

UCS503 – Software Engineering

SyncDrive

Signal-Aware Navigation and Speed Optimization System

Submitted by:

1024030177 Tarun Mahindra

1024030178 Parth Puri

1024031077 Adhiraj Kapur

Group: BE Third Year, CSE

Submitted to:

Dr. Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Implementation Details	4
2.2.1	Module 1: Green Wave Optimizer	4
2.2.2	Module 2: Normal GPS Simulation	5
3	Results and Evaluation	6
3.1	Testing Methodology	6
3.2	Performance Metrics	6
3.3	Analysis	6
4	Conclusion	8
4.1	Summary of Work	8
4.2	Key Findings	8
4.3	Future Work and Recommendations	8
4.4	Final Remarks	9

Introduction

1.1 Background and Context

Urban commuters in Indian cities spend a significant portion of their travel time waiting at traffic signals. In a city like Chandigarh, whose grid-sector layout concentrates traffic through a small number of major intersections, the stop-and-go pattern is both predictable and avoidable. Standard navigation applications—Google Maps, Apple Maps, and their equivalents—select routes based on distance and estimated travel time, but they do not account for how a driver’s chosen speed interacts with the timing of upcoming traffic signals. A driver following the fastest route at the speed limit may still stop at every signal, losing more time to red-light waits than would be saved by the speed advantage.

The concept of a *green wave*—coordinating signal timings along a corridor so that a vehicle travelling at a fixed speed passes every light on green—is well established in traffic engineering. SyncDrive inverts this concept for the individual driver: rather than the city adjusting signals to match a speed, the driver adjusts their speed and departure time to match the existing signal pattern.

1.1.1 Problem Statement

Existing navigation systems are *signal-blind*: they select a road but not the speed or departure time that would allow the driver to traverse it without stopping. This results in three compounding inefficiencies:

- **Avoidable red-light stops:** a driver who arrives at a signal thirty seconds after it turned red waits up to a full cycle (60–90 seconds) for no mechanical reason.
- **Stop-and-go fuel waste:** repeated acceleration from rest consumes significantly more fuel per kilometre than steady-speed driving.
- **No departure-time guidance:** apps tell a driver *which road* to take, not *when to leave* or *how fast to drive* on it.

The core claim of SyncDrive is that a driver travelling at a moderate, steady speed with zero red-light stops can arrive faster than one who rushes at maximum speed but accumulates waiting time at signals.

1.2 Project Objectives

1. **Objective 1—Green wave optimization:** Given a source, destination, and desired arrival time, compute the departure time and steady driving speed that min-

imises total journey time (drive time plus signal wait time) across all signals on the route.

2. **Objective 2—Quantified comparison:** Simulate a Normal GPS journey on the same route at the same departure time but at the city speed limit (50 km/h), compute the red-light wait time accumulated, and present the time saved by SyncDrive side by side.
3. **Objective 3—Working browser prototype:** Deliver a single-file web application—no install, no backend server, no API key—that renders a dark interactive map of Chandigarh, allows the user to select from 7 named locations across 42 directed routes, and displays the recommendation including departure time, speed, signal timeline, and turn-by-turn directions.

All three objectives are specific, measurable, and verifiable within the prototype: the optimizer produces a numeric departure time and speed; the comparison produces numeric time values; the browser prototype either loads and works or it does not.

Methodology and Design

2.1 System Architecture

The prototype is a single HTML file with embedded CSS and JavaScript. There is no backend, no build step, and no external API dependency beyond map tile fetching. The entire application loads and runs in the browser.

2.1.1 High-Level Design

The system has four logical layers:

1. **Data Layer**—Static JavaScript objects: **PLACES** (7 locations), **SIGNALS** (16 traffic signals), **ROUTES** (42 directed route objects).
2. **Algorithm Layer**—Two functions: `optimise()` (green wave optimizer) and `simNG()` (Normal GPS simulator).
3. **Map Layer**—Leaflet.js 1.9.4 rendering an interactive map with place labels, signal markers, route polylines, and a speed HUD.
4. **UI Layer**—Three-pane Apple Maps-inspired layout: nav rail, directions panel, map.

2.1.2 Technical Stack

- **Language:** Vanilla JavaScript (ES6+), HTML5, CSS3.
- **Map library:** Leaflet.js 1.9.4 (CDN, no install).
- **Map tiles:** Esri World Dark Gray Canvas (free, no API key).
- **Font:** Inter via Google Fonts CDN.
- **Build tools:** None—single `index.html` file.
- **Deployment:** GitHub Pages, <https://parthpuri.github.io/SyncDrive/>.
- **Version control:** Git / GitHub, <https://github.com/ParthPuri/SyncDrive>.

2.2 Implementation Details

2.2.1 Module 1: Green Wave Optimizer

The optimizer searches over a grid of candidate (speed, departure offset) pairs and returns the combination that minimises total journey time.

Algorithm:

```

Speeds : [25, 28, 30, 32, 35, 38, 40, 42, 45] km/h
Offsets: 0 to 1500 s, step 15 s

for each speed s:
    driveSec = (distKm / s) * 3600
    for each offset d:
        dep = arrivalTime - driveSec - d
        elapsed = 0
        for each signal on route:
            phase = (dep + timeToSignal + elapsed) % cycle
            if green(signal, phase): greens++
            else:
                wait = nextGreenOffset(signal, phase)
                elapsed += wait; totalWait += wait
        score = (driveSec + totalWait) - 0.5 * greens
        if score < best: save result

```

The `elapsed` accumulation is the critical correctness requirement: each red-light stop pushes the driver’s arrival at all downstream signals later, potentially converting a green into a red. Without this propagation the signal counts are wrong.

Signal phase mathematics:

- `phase = (depTime + travelToSignal + elapsed) % cycle`
- `isGreen = phase >= greenStart && phase < (greenStart + greenDuration)`
- `waitIfRed = (phase < greenStart) ? (greenStart - phase) : (cycle - phase + greenStart)`

2.2.2 Module 2: Normal GPS Simulation

The Normal GPS simulator uses the *same departure time* as the SyncDrive result but drives at 50 km/h (city speed limit). It hits signals at uncontrolled phases and accumulates real wait times using the same `elapsed` propagation logic.

Using the same departure time is essential: if the Normal GPS simulation departed later it would have less travel time and the comparison would be misleading. Equal departure time means any difference in total time is attributable solely to speed choice and signal timing—not to when the driver left.

Results and Evaluation

3.1 Testing Methodology

Testing was conducted computationally using the simulator built into the prototype. All 42 directed routes were evaluated at a 09:00 arrival target.

- **Unit testing:** Each signal phase function was verified manually against hand-computed values for known (departure, cycle, offset) combinations.
- **Integration testing:** The optimizer and simulator were run on all 42 routes; GreenWave total must never exceed Normal GPS total for the same departure.
- **Boundary testing:** Departure times near midnight were checked to confirm modular arithmetic does not produce negative phase values.
- **Browser testing:** The application was loaded in Chrome and Safari and verified to render correctly with no console errors.

3.2 Performance Metrics

Route	km	Normal GPS	SyncDrive	Saved
Sector 17 → Sector 22	1.4	3.8 min	2.6 min	1.2 min
Sector 22 → Sector 35	2.2	6.0 min	4.5 min	1.5 min
Sector 17 → PGI	3.2	5.8 min	5.0 min	0.8 min
Sector 43 → Elante Mall	2.0	4.1 min	4.0 min	0.1 min
Sector 17 → Tribune Chowk	5.1	8.2 min	7.7 min	0.5 min
Sector 22 → PGI	4.1	7.9 min	7.0 min	0.9 min
Sector 35 → Sector 43	2.8	5.9 min	4.9 min	1.0 min

Table 3.1: Journey time comparison: Normal GPS vs. SyncDrive (09:00 arrival target). All figures are based on simulated signal timings.

3.3 Analysis

Did we meet our objectives? Yes. All three objectives from Chapter 1 are fully met. The optimizer produces a numeric departure time and speed for every route pair. The comparison panel displays the time breakdown side-by-side. The single-file prototype loads in any browser with no installation.

Where did we exceed expectations? The original proposal described a single test corridor. The prototype covers 7 locations, 16 signals, and 42 directed routes—significantly

Metric	Target	Achieved
Routes covering all 7 locations	42	42
Signals modelled	≥ 10	16
SyncDrive beats or ties Normal GPS	100%	100%
Application loads without server	Yes	Yes
No API key required	Yes	Yes

Table 3.2: Prototype objective achievement summary.

broader than scoped. The UI additionally renders a per-signal timeline, turn-by-turn directions, and a speed HUD.

Challenges encountered and resolved:

- **Tile provider API keys:** Both Carto Voyager and Stadia Maps began requiring API keys after project start. Resolved by switching to Esri ArcGIS Dark Gray Canvas, which remains freely accessible.
- **Wrong optimization objective:** The first version maximised green signal count, ignoring total journey time. Fixed by minimising $(t_{drive} + t_{wait})$.
- **Missing elapsed propagation:** Early builds did not accumulate wait time through the signal loop. Fixed by threading `elapsed` through the simulation.
- **Unfair comparison baseline:** The Normal GPS simulation initially used the arrival time as its departure reference. Fixed to use the same departure time as SyncDrive.

Conclusion

4.1 Summary of Work

SyncDrive demonstrates that a driver can arrive at a destination faster by driving at a lower, steady speed timed to a signal wave, compared to rushing at the city speed limit and accumulating red-light wait time. The prototype delivers a working single-file web application covering Chandigarh’s key corridors, with an interactive dark map, 42 routes, 16 simulated signals, a green wave optimizer, a Normal GPS comparison, and an Apple Maps-inspired interface—all without a backend server or API key.

4.2 Key Findings

- **Finding 1—Slower speed, faster arrival:** On short routes (1–2 km), driving at 30–35 km/h with zero stops consistently beats 50 km/h with 2–3 red-light stops by 0.5–1.5 minutes.
- **Finding 2—Elapsed propagation is non-negotiable:** The single most important correctness requirement is accumulating wait time through the signal loop. Omitting it produces systematically optimistic green counts.
- **Finding 3—Fair comparison requires equal departure time:** Using the same departure time for both strategies isolates the effect of speed and signal timing, making time savings directly attributable to the green wave approach.

4.3 Future Work and Recommendations

1. **Live position tracking:** Use the browser Geolocation API to track the driver’s real-time position and continuously recalculate the recommended speed [OSM26].
2. **Departure countdown:** Add a “Leave in X minutes” live timer based on the current system clock.
3. **Animated route playback:** Move a vehicle marker along the polyline at the recommended speed.
4. **Real signal data:** Source signal cycle lengths from OpenStreetMap `highway=traffic_signals` and `cycle_time:*` tags [OSM26].
5. **Peak-hour profiles:** Model different signal cycle lengths for morning (08:00–10:00) and evening (17:00–20:00) rush hours.
6. **OSRM routing integration:** Replace hardcoded polylines with road-following routes from the OSRM open-source routing engine.

7. **Emergency vehicle coordination:** Phase 2 feature from the original proposal—adjust nearby drivers’ speed recommendations when an emergency vehicle is active on the same corridor.

4.4 Final Remarks

The prototype validates the central hypothesis of SyncDrive within a controlled, simulated environment. The engineering challenge was not the map rendering or the UI—it was getting the signal phase simulation mathematically correct and ensuring the comparison between strategies was scientifically fair. Both issues were identified, debugged, and fixed during the five weeks of development documented in the team journals. The foundation is solid for the next phase of work.